

Numérique et Sciences informatiques	Première
Gestion de fichiers	

1) Création de fichiers et de répertoires

2.1) Création de fichiers dans le répertoire courant et sous-répertoire

Pour comprendre comment lire un fichier en python, nous allons créer un fichier dans le répertoire de travail.

Exercice 1

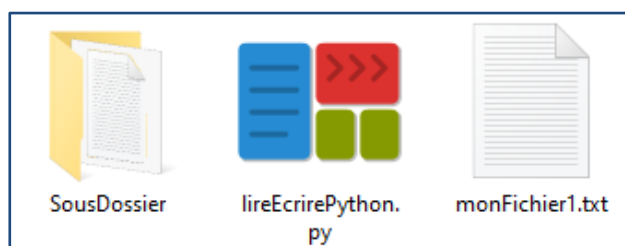
- (1) Créer un fichier python **lireEcrirePython.py** dans un répertoire (= dossier) de travail.

Pyzo a besoin de savoir dans quel dossier l'on travaille pour la suite du TP. Pour cela : exécuter une première fois votre fichier (même vide) à l'aide de la commande : **Démarrer le script** du menu **Exécuter**.

- (2) Ouvrir l'éditeur de texte Bloc-notes.
- (3) Créer un fichier **monFichier1.txt** dans le même dossier et dans lequel sera écrit "Hello world !"
- (4) Puis créer un nouveau fichier **monFichier2.txt** qui sera enregistré dans un nouveau dossier nommé **sousDossier**.

On écrira le texte suivant : "Ceci est le texte du fichier 2 du sous-dossier!".

Voilà ce que doit contenir votre répertoire de travail :



2.2) Chemin relatif / chemin absolu

Un chemin relatif en informatique est un chemin qui prend en compte l'emplacement de lecture.

Si on exécute le programme créé par **lireEcrirePython.py** à partir du dossier courant,

- Le chemin vers **monFichier1.txt** est simplement le nom du fichier **monFichier1.txt**.
- Le chemin vers **monFichier2.txt** est **sousDossier/monFichier2.txt**.

Un chemin absolu est un chemin complet qui peut être lu quel que soit l'emplacement de lecture.

Si on exécute le programme créé par **lireEcrirePython.py** à partir d'un dossier quelconque, Il faut connaître le nom du répertoire racine (Souvent **C:** sous Windows) et indiquer le chemin vers le fichier. Par exemple :

- Le chemin vers **monFichier1.txt** est **C:/NSI/gestionDeFichiers/monFichier1.txt**.
- Le chemin vers **monFichier2.txt** est **C:/NSI/gestionDeFichiers/sousDossier/monFichier2.txt**.

Dans la pratique on utilisera très peu le chemin absolu.

2) Ouverture et fermeture de fichiers

2.1) Ouverture de fichiers

```
>>> fichier1 = open("monFichier1.txt", "r")

>>> print(fichier1)
<_io.TextIOWrapper name='monFichier1.txt' mode='r' encoding='cp1252'>
```

Le résultat n'est pas le contenu du fichier mais des informations sur la variable **fichier1**. C'est ce qu'on appelle un pointeur qui va pointer sur les caractères contenus dans le fichier **monFichier1.txt**.

Il existe plusieurs modes d'ouverture qui peuvent être :

- **"r"**, pour une ouverture en lecture (READ).
- **"w"**, pour une ouverture en écriture (WRITE), à chaque ouverture le contenu du fichier est écrasé.
Si le fichier n'existe pas, python le crée.
- **"a"**, pour une ouverture en mode ajout à la fin du fichier (APPEND). Si le fichier n'existe pas python le crée.
- **"b"**, pour une ouverture en mode binaire.
- **"x"**, crée un nouveau fichier et l'ouvre pour écriture.

encoding='cp1252' correspond à l'encodage des caractères.

Exercice 2

Ecrire la ligne de commande pour ouvrir le fichier **monFichier2.txt** en mode lecture.

2.2) Fermeture de fichiers

Comme tout élément ouvert, il faut le refermer une fois les instructions terminées. On utilise alors la méthode **close()**.

```
>>> fichier1.close()

>>> fichier2.close()
```

3) Lecture et écriture

3.1) Lire le contenu d'un fichier

Un fichier texte est un fichier qui contient des caractères imprimables et des espaces organisés en lignes successives, ces lignes étant séparées les unes des autres par un caractère spécial non-imprimable appelé « marqueur de fin de ligne ».

Les fichiers textes sont donc des fichiers que nous pouvons lire et comprendre à l'aide d'un simple éditeur de texte, par opposition aux fichiers binaires dont le contenu est - au moins en partie - inintelligible pour un lecteur humain, et qui ne prend son sens que lorsqu'il est décodé par un logiciel spécifique.

Par exemple, les fichiers contenant des images, des sons, des vidéos, etc. sont presque toujours des fichiers binaires.

Pour afficher tout le contenu d'un fichier, vous pouvez utiliser la méthode **read()** sur l'objet-fichier.



```
1 fichier1 = open("monFichier1.txt", "r")
2 print (fichier1.read())
3 fichier1.close()
```

3.2) Écrire dans un fichier : mode ajout 'a' ou mode écriture 'w'

Il est très facile de traiter des fichiers textes avec Python. Il existe pour cela deux modes d'écriture (ajout ou écriture).

3.2.1) Mode écriture 'w' (write) : tout l'ancien contenu est écrasé.

Par exemple, les instructions suivantes suffisent pour créer un fichier texte de quatre lignes :

 Python

```
1 f = open("monFichier1.txt", "w") # mode w donc l'ancien contenu est effacé
2 f.write("Ceci est la ligne un\nVoici la ligne deux\n")
3 f.write("Voici la ligne trois\nVoici la ligne quatre\n")
4 f.close()
```

Bien noter le marqueur de fin de ligne `\n` inséré dans les chaînes de caractères, aux endroits où l'on souhaite séparer les lignes de texte dans l'enregistrement. Sans ce marqueur, les caractères seraient enregistrés les uns à la suite des autres.

Les lignes de code suivantes permettent l'affichage du contenu du fichier :

 Python

```
5 f = open("monFichier1.txt", "r") # ouverture en mode lecture
6 print(f.read())
7 f.close()
```

3.2.2) Mode ajout 'a' (append) : on ajoute à l'ancien contenu.

Pour ajouter du texte à ce qui précède :

 Python

```
1 f = open("monFichier1.txt", "a") # mode append a donc ajout au contenu
2 f.write("Bonjour monde")
3 f.close()
```

Regarder maintenant le contenu de `monFichier1.txt`.

 Python

```
4 f = open("monFichier1.txt", "r") # ouverture en mode lecture
5 print(f.read())
6 f.close()
```

4) Quelques autres commandes

4.1) Le mot clé `with`

Il existe une autre syntaxe plus courte qui permet de s'émanciper du problème de fermeture du fichier : le mot clé `with`.

Voici la syntaxe (noter l'indentation qui permet de savoir quand fermer le fichier) :

 Python

```
1 with open("monFichier1.txt", "r") as fichier1 :
2     print (fichier1.read())
3
4 # Cela équivaut en fait à l'écriture :
5
6 fichier1 = open("monFichier1.txt", "r")
7 print (fichier1.read())
```

4.2) Lecture ligne par ligne

4.2.1) La méthode `readline()` : ligne par ligne

Lors des opérations de lecture, les lignes d'un fichier texte peuvent être extraites séparément les unes des autres.

La méthode `readline()`, par exemple, ne lit qu'une seule ligne à la fois (en incluant le caractère de fin de ligne) :

```
>>> f = open('monFichier1.txt', 'r')
```

```
>>> t = f.readline()
```

```
>>> print(t)
Ceci est la ligne un
```

```
>>> print(f.readline())
Voici la ligne deux
```

A chaque appel, on va lire une ligne du fichier. Pour afficher les lignes une par une il faut donc effectuer une boucle et tester si la ligne extraite est vide.



```
1 f = open('monFichier1.txt', 'r')
2 while True: # c'est une boucle sans fin, on utilise un break pour en sortir
3     txt = f.readline()
4     if txt == '' :
5         break
6     print(txt)
7 f.close()
```

Exercice 3

Modifier le programme précédent afin de ne pas utiliser d'instruction `break`.

4.2.2) La méthode `readlines()` : toutes les lignes restantes

Lors des opérations de lecture, les lignes d'un fichier texte peuvent être extraites séparément les unes des autres.

La méthode `readlines()` (noter le **s** à la fin du mot `readlines`) transfère toutes les lignes restantes dans une liste de chaînes :

```
>>> f = open('monFichier1.txt', 'r')
```

```
>>> t = f.readline()
```

```
>>> print(t)
Ceci est la ligne un
```

```
>>> print(f.readline())
Voici la ligne deux
```

```
>>> print(f.readlines())
['Voici la ligne trois\n', 'Voici la ligne quatre\n', 'Bonjour monde']
```

```
>>> f.close()
```

Remarques :

- La liste apparaît ci-dessus en format brut, avec des apostrophes pour délimiter les chaînes, et les caractères spéciaux sous leur forme conventionnelle. On pourra bien évidemment parcourir cette liste (à l'aide d'une boucle **while**, par exemple) pour en extraire les chaînes individuelles.
- La méthode **readlines()** permet donc de lire l'intégralité d'un fichier en une instruction seulement. Cela n'est possible toutefois que si le fichier à lire n'est pas trop gros : puisqu'il est copié intégralement dans une variable, c'est-à-dire dans la mémoire vive de l'ordinateur, il faut que la taille de celle-ci soit suffisante.
- Si l'on doit traiter de gros fichiers, on utilisera plutôt la méthode **readline()** dans une boucle.
- Bien noter que **readline()** est une méthode qui renvoie une chaîne de caractères, alors que la méthode **readlines()** renvoie une liste. À la fin du fichier, **readline()** renvoie une chaîne vide, tandis que **readline()** renvoie une liste vide.

4.3) Créer un nouveau répertoire

Pour créer un nouveau répertoire dans le répertoire courant :

```
 Python
1 import os
2 os.mkdir('monDossierAMoi')
```

5) Exercices

A chaque fois que l'on demande d'ouvrir ou d'écrire, il faut le faire en python.

Exercice 4

Programmer l'algorithme suivant en python :

- (1) Ouvrir le fichier **monFichier2.txt**.
- (2) Afficher le contenu de **monFichier2.txt**.
- (3) Créer un sous-répertoire **monDossier**.
- (4) Écrire le contenu de **monFichier2.txt** dans un fichier **monFichier3.txt** et le mettre dans le dossier **monDossier**.

Exercice 5

Cet exercice est un simple exercice de manipulation de fichiers pour comprendre la différence entre les modes d'ouverture de fichiers : **append** (ajouter) '**a**', **write** (écrire) '**w**' et **read** (lire) '**r**'.

- (1) Créer le fichier **exercice5.txt** en l'ouvrant en mode écriture, écrire un caractère dedans, puis le refermer.
- (2) L'ouvrir en mode lecture, afficher ce qui est lu dedans, puis le refermer.
- (3) L'ouvrir en mode écriture, écrire "Hello " dedans, puis le refermer.
- (4) L'ouvrir en mode lecture, afficher ce qui est lu dedans, puis le refermer. Que remarque-t-on ?
- (5) L'ouvrir en mode ajout, écrire "World !" dedans, puis le refermer.
- (6) L'ouvrir en mode écriture, afficher ce qui est lu dedans, puis le refermer. Que remarque-t-on ?

Exercice 6

Créer une fonction **nbrLigne(fichier)** qui a pour paramètre le nom d'un fichier (texte) et qui renvoie le nombre de lignes de ce fichier. Il est possible de créer un fichier avec plusieurs lignes pour tester le programme.

La question est de savoir quand le fichier est terminé.

Exercice 7

- (1) Avant de commencer l'exercice, copier et enregistrer le fichier **demi.txt** dans le répertoire de travail.

Voici une chaîne de caractères :

" La mesure de l'homme. \n \n Ce n'est pas celui qui critique qui est important, ni celui qui montre du doigt comment l'homme fort trébuche ou comment l'homme d'action aurait pu faire mieux.\n L'hommage est dû à celui ou à celle qui se bat dans l'arène, dont le visage est couvert de poussière et de sueur, qui va de l'avant vaillamment, qui commet des erreurs et en commettra encore, car il n'y a pas d'efforts humains sans erreurs et imperfections. C'est à lui ou à elle qu'appartient l'hommage, à celui ou à celle dont l'enthousiasme et la dévotion sont grands, à celui ou à celle qui se consume pour une cause importante, à celui ou à celle qui, au mieux, connaîtra le triomphe du succès, et au pis, s'il échoue, saura qu'il a échoué alors qu'il risquait courageusement."

L'exercice consiste à :

- (2) Créer un autre fichier texte nommé **laMesureDeLHomme.txt** et écrire la chaîne de caractères dedans.
(Copier la chaîne dans l'éditeur python puis écrire le programme pour l'insérer dans **laMesureDeLHomme.txt**)
- (3) Écrire le contenu du fichier **demi.txt** à la suite du fichier **laMesureDeLHomme.txt**.
- (4) Afficher le contenu du fichier **laMesureDeLHomme.txt**.

Exercice 8 La disparition ...

- (1) Écrire une fonction **nombreOccurrence(lettre, texte)** qui compte le nombre d'occurrences de la variable **lettre** dans le **texte** qui est une chaîne de caractères.
- (2) Effectuer des tests de la fonction avec une lettre et des chaînes de caractères importées via les fichiers existants.
- (3) Importer maintenant le fichier texte **laDisparition.txt** qui propose un extrait du roman « La Disparition » écrit en 1968 par l'écrivain français Georges Perec (1936-1982) et publié en 1969.
- (4) Essayer la fonction de recherche sur ce texte avec quelques mots comme **'je', 'la', 'les', 'envie', 'esprit', 'est', 'les'**. Puis avec la lettre **'e'**, Que remarque-t-on ?
- (5) Écrire une fonction qui renvoie le nombre de lignes et le nombre de caractères de votre texte.